MECHATRONICS · UAV SYSTEMS · STUDENT RESEARCH

TriFlight Initiative

Avionics Redesign and Systems-Integration Study for a Modular Long-Endurance Fixed-Wing UAV

AUTHOR	Amaan Nizam (Sahil)
INSTITUTION	THWS
PROGRAMME	B.Eng. Mechatronics Engineering
DOCUMENT TYPE	Pre-Experimental Research & Design Report
DATE	April 2026

Project Status Notice

This is a pre-experimental design report. All research, architecture, methodology, and software work is complete. Physical implementation was halted solely due to the absence of hardware funding. No results have been fabricated. All data tables are intentionally marked as pending.

Abstract

The TriFlight Initiative is a student-led mechatronics research project focused on the avionics modernisation of a modular long-endurance fixed-wing unmanned aerial vehicle. The reference airframe the Flightory Talon 1400 is treated as a credited baseline platform; no claim of original mechanical authorship is made. The engineering contribution of TriFlight lies entirely in the avionics layer: a redesigned power and signal architecture, an expanded sensor stack, flight-controller selection and firmware configuration strategy, a Python-based MAVLink telemetry logger, and a structured bench-to-flight validation methodology.

This document is classified as a pre-experimental design report. The research methodology, component trade studies, wiring architecture, firmware analysis, and Python implementation are fully developed. Empirical bench and flight measurements could not be collected due to funding and hardware access constraints. All blank result tables are intentionally preserved as honest placeholders, ready for population upon resource availability.

The document demonstrates that a mechatronics student, working independently and without laboratory funding, can produce a technically rigorous avionics architecture, a justified component selection, working telemetry software, and a complete validation plan for a fixed-wing UAV system.

Keywords

Fixed-wing UAV · ArduPilot · INAV · MAVLink · pymavlink · sensor integration · power electronics · telemetry · V-tail mixing · servo mapping · systems engineering · mechatronics · pre-experimental research

Contents

1.Introduction
2.Research Problem and Scope
3.Baseline Architecture Study
4.Research Questions
5.System Requirements
6.Methodology
7.Component Selection and Trade Study
8.Proposed Avionics Architecture
9.Power System and Signal Architecture
10.Sensor Expansion Strategy
11.Firmware and Control-System Selection
12.MAVLink Telemetry Logger: Design and Implementation
13.Bench-Test Protocol
14.Virtual Testing and Simulation Plan
15.Flight-Test Protocol
16.Risk, Safety, and Regulatory Compliance
17.Results Framework and Data Sheets
18.Discussion and Resource Constraint Statement
19.Development Roadmap
20.Attribution and Research Ethics
21.References
Appendix A — Pin Mapping Template
Appendix B — Minimum Evidence Package

1. Introduction

Small fixed-wing unmanned aerial vehicles are among the most demanding educational platforms in mechatronics engineering. They require simultaneous integration of mechanical understanding, electrical power systems, embedded firmware, control theory, wireless communication, and systematic test procedures. Unlike a standalone CAD model or a standalone circuit exercise, a fixed-wing UAV exposes integration failures immediately: an incorrect centre of gravity produces instability during launch, an incorrect servo direction produces immediate loss of control authority, incorrect voltage routing destroys electronic components, and poor wiring practice produces intermittent faults that are difficult to diagnose.

The TriFlight Initiative is a structured student research and development project rather than a claim of original airframe invention. The project studies a modular endurance-class reference UAV platform the Flightory Talon 1400 and focuses all original work on the avionics layer: wiring architecture, sensor selection and integration, firmware configuration logic, telemetry software, and validation methodology. This is a technically demanding and intellectually legitimate engineering scope because the majority of real-world UAV failures originate not from external airframe geometry, but from incorrect integration of power rails, signal paths, firmware parameters, and operator procedures.

The project was designed to be completed with physical hardware. Implementation was halted at the design and software stage due to the absence of a hardware budget. This report documents everything that was completed: the research, the architectural design, the software implementation, and the full validation plan. It is submitted as evidence of engineering capability and readiness, not as a fabricated experimental result.

2. Research Problem and Scope

The core research problem is: how can a student engineer convert a minimal fixed-wing UAV wiring architecture into a measurable, testable, and research-ready avionics system, and what methodology and validation process should govern that conversion?

The scope is intentionally focused. The mechanical platform remains close to the reference design because the claimed contribution is specific and honest: TriFlight does not claim original authorship of the airframe geometry. It claims a planned avionics redesign — selecting the flight controller, defining signal and power routing, integrating sensors, writing telemetry software, and establishing a staged validation process.

2.1 In-Scope Work

- Flight-controller platform selection with documented trade study
- ESC, power module, and current/voltage sensing integration
- Servo signal mapping for aileron and V-tail control surfaces
- GPS, compass, airspeed sensor, and barometer integration strategy
- Telemetry radio integration and MAVLink data pipeline
- Python MAVLink logger implementation using pymavlink
- Mission Planner / QGroundControl configuration strategy
- Bench test protocol, preflight checklist, and flight-test plan
- Risk assessment and regulatory compliance framework

2.2 Out-of-Scope Work

- Original authorship of the Talon 1400 airframe CAD geometry
- Fabrication of bench or flight measurements not yet collected
- Autonomous flights beyond regulatory and institutional approval
- Payloads that create safety, legal, or dual-use risk

3. Baseline Architecture Study

The Talon 1400 baseline wiring concept uses a conventional fixed-wing control layout: two aileron servos for roll control, two V-tail servos for mixed pitch and yaw authority, one ESC-driven throttle channel, a flight controller, GPS, and a battery-to-ESC power path. In a basic flight configuration this is sufficient to make the aircraft controllable. However it does not constitute a research-ready platform because it lacks explicit measurement channels and documented validation procedures.

Channel	Assignment	Device	Notes
S1	Left aileron	Wing servo (left)	Roll authority; direction must be verified
S2	Right aileron	Wing servo (right)	Opposite deflection to S1 for roll
S3	V-tail left	Ruddervator servo (left)	Mixed pitch and yaw
S4	V-tail right	Ruddervator servo (right)	Mixed pitch and yaw, opposite for yaw
S5	Throttle / ESC	Electronic speed controller	Motor direction test first, prop removed

Table 1. Baseline actuator channel mapping retained by TriFlight as logical starting point.

The most important limitation of the baseline is not that it is incorrect; it is that it is minimal. A minimal layout can fly, but a research platform must measure. TriFlight addresses five specific gaps: explicit power monitoring, servo rail independence, airspeed sensing, telemetry logging, and a structured validation workflow.

4. Research Questions

ID	Research Question	Required Evidence
RQ1	Can a baseline fixed-wing wiring layout be extended into a research-grade avionics architecture without major airframe redesign?	Revised wiring diagram, component table, bench checklist
RQ2	Does the proposed sensor stack provide sufficient data to evaluate power, navigation, and flight-control behaviour?	MAVLink logs: GPS, battery, attitude, airspeed fields
RQ3	Which firmware is most suitable for a student fixed-wing project: ArduPilot, INAV, or PX4?	Decision matrix and configuration complexity comparison
RQ4	Can a Python MAVLink logger capture and persist useful telemetry for post-flight analysis?	Working script, sample CSV, timestamped log fields
RQ5	What bench-test sequence is required before a safe maiden flight?	Continuity, polarity, servo direction, power, GPS, failsafe records

Table 2. Research questions and required evidence.

5. System Requirements

Requirements are divided into functional, electrical, software, and safety categories. A component is only selected if it satisfies a defined requirement. This prevents the project from becoming an unstructured parts list.

ID	Requirement	Verification Method
FR-1	Aircraft shall support four control-surface outputs: left aileron, right aileron, left V-tail, right V-tail.	Servo movement test through flight controller
FR-2	Aircraft shall support throttle control through ESC signal output.	Motor direction and throttle-response bench test
FR-3	System shall support at least one stabilised mode and one manual fallback mode.	Firmware configuration review and dry test
FR-4	System shall support return-to-launch or comparable failsafe if GPS and firmware support it.	Failsafe configuration test without propeller
ER-1	Battery voltage and current shall be measured via a power module.	Mission Planner battery telemetry screen
ER-2	5V rail shall supply receiver, FC logic, telemetry, GPS, and servos within current limit.	Bench current measurement under servo load
SR-1	All ground references shall be common between FC, ESC signal, receiver, telemetry, and sensors.	Continuity test
SW-1	Telemetry logger shall record time, voltage, current, GPS, altitude, mode, and attitude.	CSV log field inspection
SAFE-1	No motor test shall be performed with propeller installed before servo and failsafe checks.	Safety checklist signature

Table 3. System requirements and verification methods.

6. Methodology

The methodology follows a design-validate-iterate structure and stops explicitly short of claiming experimental results before measurements exist. The workflow is: baseline analysis → requirements definition → component selection → schematic redesign → firmware selection → telemetry software implementation → bench testing → virtual testing → field testing → log analysis → iteration.

Stage	Activity	Deliverable	Status
1	Continuity and polarity verification	Multimeter checklist	Pending hardware
2	Servo response and direction check	Control surface verification log	Pending hardware
3	Power bench test — rail load and sag	Voltage and current table	Pending hardware
4	Sensor calibration — GPS, compass, airspeed	Calibration screenshots	Pending hardware
5	Ground range check and failsafe test	RC and telemetry link report	Pending hardware
6	Maiden flight — manual or stabilised	Flight log and inspection notes	Pending hardware
7	Log analysis and paper revision	Updated results section	Pending hardware

Table 4. Seven-stage validation pipeline. A maiden flight is stage six, not stage one.

6.1 Baseline Analysis

The original minimal wiring layout is translated into a formal signal map. The objective is to preserve correct actuator logic while adding measurement and safety layers. Output assignments are formalised: S1 left aileron, S2 right aileron, S3 left V-tail, S4 right V-tail, S5 throttle/ESC.

6.2 Redesign Strategy

The redesign is limited to the avionics and systems integration layer. The primary improvement is a measurable electrical architecture: battery current and voltage sensing via a power module, independent servo power through a UBEC, telemetry output, optional airspeed sensing, and a logging pipeline. This creates a platform capable of producing data for future empirical versions of this report.

7. Component Selection and Trade Study

Component selection is based on six weighted criteria: fixed-wing hardware support, firmware compatibility, documentation quality, integration risk, student procurement cost, and measurement capability. A high-performance component that is poorly documented may be worse for a student research project than a simpler component with an excellent support ecosystem.

Subsystem	Recommended Option	Alternative	Reason for Selection
Flight controller	SpeedyBee F405 Wing / Matek F405 Wing	Pixhawk-class FC	Wing-specific output layout, compact, excellent ArduPilot/INAV support
Firmware	ArduPilot Plane	INAV	MAVLink logging, RTL, autonomous missions, research depth
RC receiver	ELRS receiver	FrSky/SBUS	Low latency, long range, strong open-source community
GPS / compass	M10Q-class GPS with integrated compass	Basic GPS only	Improved navigation accuracy and heading estimation
Airspeed sensor	Digital pitot sensor (MS4525DO or equivalent)	None	Stall awareness and autonomous speed control for fixed-wing
Power module	Current and voltage sensing power module	Voltage divider only	Measures energy consumption, not just voltage
Telemetry	SiK radio / ELRS telemetry / Wi-Fi bridge	No telemetry	Real-time supervision and CSV data capture
Ground software	Mission Planner / QGroundControl	Manual transmitter	Parameter setup, log download, mission planning
Logger	Python pymavlink script (custom)	Built-in FC log only	Custom CSV fields, demonstrates software capability

Table 5. Component selection table with justification.

7.1 Firmware Trade Study

Firmware selection determines the entire scope of autonomous capability, telemetry depth, and configuration complexity. Three options were evaluated:

Criterion	ArduPilot Plane	INAV	PX4
Fixed-wing autonomy	Strong: missions, RTL, TECS	Moderate: simpler nav stack	Strong but complex
Learning curve	Moderate	Low	High
Telemetry ecosystem	Strong MAVLink	Good, less research depth	Strong MAVLink
Logging capability	Comprehensive .bin logs	Limited vs ArduPilot	Comprehensive ULog
Student suitability	Best for research logs	Best for quick first flight	Advanced teams
TriFlight recommendation	Primary firmware	Backup option	Future exploration

Table 6. Firmware comparison matrix. ArduPilot Plane is recommended for the research configuration.

8. Proposed Avionics Architecture

The proposed architecture adds explicit sensing and telemetry around the baseline flight-control wiring. The flight controller remains the central decision point. Six architectural improvements are made relative to the baseline: (1) power module with current and voltage sensing, (2) independent UBEC servo rail, (3) GPS with integrated compass, (4) airspeed sensor, (5) telemetry radio, and (6) Python-based ground-side logging layer.

Block	Component	Interface	Function
4S Li-Ion battery	4S–6S pack, 18650 or LiPo	XT60 connector	Main energy source
Power module	Current + voltage sensor	Inline VBAT, analog to FC	Power monitoring
Flight controller	SpeedyBee / Matek F405 Wing	Central hub	Stabilisation and mission computer
Independent UBEC	5V 3A+ BEC	5V rail to servos	Servo power isolated from FC regulator
ESC + motor	Suitable for 1400mm platform	PWM / DShot from S5	Propulsion
Servos S1–S4	Standard wing servos x 4	PWM from FC + 5V rail	Ailerons and V-tail
GPS + compass	M10Q or equivalent	UART + I2C	Position, heading, autonomous nav
Airspeed sensor	Digital pitot (I2C)	I2C to FC	Airflow measurement, stall awareness
Telemetry radio	SiK 915/433 MHz or ELRS	UART MAVLink	Real-time GCS link
RC receiver	ELRS receiver	CRSF/SBUS to FC	Pilot command input
Companion logger	Laptop running Python script	USB / UDP MAVLink	CSV telemetry capture

Table 7. Proposed avionics architecture blocks with interface and function.

9. Power System and Signal Architecture

A fixed-wing UAV contains two different electrical realities that must not be mixed without careful design: high-current propulsion power and low-current control electronics. Mixing these incorrectly creates noise on sensor lines, voltage brownouts during throttle transients, and component failure from reverse polarity or overcurrent.

Connection	Voltage / Signal	Purpose	Failure Mode if Incorrect
Battery to ESC	4S: 14.8V nominal, 16.8V max	Propulsion power	ESC damage, connector overheating
Battery to power module	4S VBAT	Voltage/current measurement	No battery telemetry or wrong readings
Power module/BEC to FC	5V regulated	FC logic power	Brownout or FC reset in flight
FC to ESC	PWM or DShot throttle signal	Throttle command	No throttle or unsafe throttle response
FC to servos	PWM signal + 5V/GND rail	Control surface movement	Wrong motion or servo overload
GPS to FC	UART/I2C + 5V/GND	Position and heading	No navigation or poor heading
Telemetry to FC	UART + 5V/GND	MAVLink uplink/downlink	No ground monitoring
Receiver to FC	CRSF or SBUS + 5V/GND	Pilot command input	No pilot control

Table 8. Power and signal connections with failure mode analysis.

Key routing rules: (1) high-current ESC and motor wires must be kept physically separate from compass and GPS wiring to minimise magnetic interference; (2) all grounds must be common; (3) the ESC must receive airflow for cooling; (4) telemetry antennas must be positioned away from carbon fibre reinforcement and high-current conductors.

10. Sensor Expansion Strategy

The baseline sensor set contains only GPS. TriFlight expands the sensor stack to answer flight-safety and endurance questions that the baseline cannot address. The most useful additional sensors are not exotic; they are practical instruments with well-understood interfaces.

Sensor	Interface	Engineering Value	Key Research Variable
GPS	UART	Position, groundspeed, home point, autonomous nav	3D fix, HDOP, satellite count
Compass	I2C (integrated with GPS)	Heading reference for navigation	Heading consistency, interference
Current sensor	Analog/I2C via power module	Real energy consumption measurement	Cruise and peak current, energy
Voltage sensor	Analog via power module	Battery health and failsafe trigger	Voltage sag under load
Airspeed sensor	I2C (MS4525DO)	Fixed-wing stall margin, speed control	Airspeed vs groundspeed in wind
Barometer	Integrated in FC	Altitude estimation	Altitude drift, pressure stability
IMU (6-axis)	Integrated in FC	Attitude stabilisation	Roll/pitch/yaw response, vibration

Table 9. Sensor expansion rationale. Airspeed receives particular attention because lift depends on airflow, not GPS groundspeed.

11. Firmware and Control-System Selection

ArduPilot Plane is recommended as the primary firmware for TriFlight. It has mature fixed-wing support through the TECS (Total Energy Control System) speed and altitude controller, mission planning via waypoints, return-to-launch failsafe, geofencing, and comprehensive MAVLink telemetry with full .bin log export. INAV remains a practical alternative for the first flying prototype because it has a lower configuration barrier. PX4 is credible and used in professional research UAVs, but adds unnecessary complexity at this project stage.

Output	Function	Physical Device	Verification Test
S1	Left aileron	Wing servo (left)	Right-stick roll: left aileron up, right aileron down
S2	Right aileron	Wing servo (right)	Opposite deflection to S1; verify in stabilise mode
S3	V-tail left	Ruddervator servo (left)	Pitch and yaw mix verified with stick inputs
S4	V-tail right	Ruddervator servo (right)	Pitch and yaw mix verified; opposite yaw direction
S5	Throttle	ESC signal	Motor test without prop first; confirm direction before installing prop

Table 10. Proposed output mapping and verification test for each channel.

12. MAVLink Telemetry Logger: Design and Implementation

A research project requires recorded data. Mission Planner and QGroundControl save binary .bin logs, but a custom Python logger demonstrates software engineering capability and produces a human-readable CSV that can be opened in any analysis tool. The implementation below is functional, not pseudocode. It requires only the pymavlink library and runs over serial or UDP.

```
# TriFlight MAVLink Telemetry Logger
# Requirements: pip install pymavlink
# Serial usage:  python triflight_logger.py --conn COM7 --baud 57600
# SITL usage:    python triflight_logger.py --conn udp:127.0.0.1:14550

import argparse, csv, time
from datetime import datetime, timezone
from pymavlink import mavutil

FIELDS = [
    "utc_time", "mode", "lat_deg", "lon_deg", "relative_alt_m",
    "groundspeed_mps", "airspeed_mps", "roll_deg", "pitch_deg", "yaw_deg",
    "battery_voltage_v", "battery_current_a", "battery_remaining_pct"
]

deg_from_e7  = lambda v: None if v is None else v / 1e7
m_from_mm    = lambda v: None if v is None else v / 1000.0
deg_from_rad = lambda v: None if v is None else v * 57.2957795

def main():
    parser = argparse.ArgumentParser(description='TriFlight MAVLink CSV logger')
    parser.add_argument('--conn', required=True)
    parser.add_argument('--baud', type=int, default=57600)
    parser.add_argument('--out',  default='triflight_telemetry.csv')
    args = parser.parse_args()

    master = mavutil.mavlink_connection(args.conn, baud=args.baud)
    print('Waiting for heartbeat...')
    master.wait_heartbeat()
    print(f'Connected: system {master.target_system}')

    state = {f: None for f in FIELDS}
    last_write = 0

    with open(args.out, 'w', newline='') as f:
        writer = csv.DictWriter(f, fieldnames=FIELDS)
```

```
        writer.writeheader()
        while True:
            msg = master.recv_match(blocking=True, timeout=1)
            if msg is None: continue
            state['utc_time'] = datetime.now(timezone.utc).isoformat()
            state['mode']     = master.flightmode
            t = msg.get_type()
            if t == 'GLOBAL_POSITION_INT':
                state['lat_deg']        = deg_from_e7(msg.lat)
                state['lon_deg']        = deg_from_e7(msg.lon)
                state['relative_alt_m'] = m_from_mm(msg.relative_alt)
                state['groundspeed_mps']= ((msg.vx**2 + msg.vy**2)**0.5) / 100.0
            elif t == 'VFR_HUD':
                state['airspeed_mps']   = msg.airspeed
            elif t == 'ATTITUDE':
                state['roll_deg']       = deg_from_rad(msg.roll)
                state['pitch_deg']      = deg_from_rad(msg.pitch)
                state['yaw_deg']        = deg_from_rad(msg.yaw)
            elif t == 'BATTERY_STATUS':
                vv = [v for v in msg.voltages if v != 65535]
                if vv: state['battery_voltage_v'] = sum(vv) / 1000.0
                if msg.current_battery  != -1:
                    state['battery_current_a']     = msg.current_battery / 100.0
                if msg.battery_remaining != -1:
                    state['battery_remaining_pct'] = msg.battery_remaining
            if time.time() - last_write >= 0.5:
                writer.writerow(state); f.flush()
                last_write = time.time()

if __name__ == '__main__':
    main()
```

Listing 1. TriFlight MAVLink telemetry logger — fully executable Python implementation.

The logger is intentionally non-intrusive: it only listens and records; it never sends commands to the flight controller. This makes it safe to run during bench tests and flight operations. Later versions can add low-battery warnings, geofence alerts, or a real-time dashboard.

13. Bench-Test Protocol

Bench tests convert the paper from a proposal into a documented experiment. The propeller must be removed before any electronic configuration work. Each test must be photographed or logged. Result columns are intentionally blank fabricating measurements would be dishonest and would undermine the research.

Test	Procedure	Pass Criteria	Result
Continuity	Multimeter: confirm all grounds common, no VBAT-to-GND short	No short; common GND confirmed	Pending
Polarity	Check battery connector, ESC, power module, 5V rail	Correct polarity on every connector	Pending
5V rail idle	Power system without servos moving; measure 5V rail	4.9–5.2 V stable	Pending
Servo load	Move all surfaces 60 s; measure voltage drop and current	No brownout; rail stable	Pending
ESC signal	Remove propeller, arm, increase throttle slowly	Motor responds smoothly	Pending
GPS lock	Power aircraft outdoors; record time to 3D fix	Stable fix, HDOP < 2.0	Pending
Failsafe	Simulate RC loss with prop removed	Expected failsafe mode triggers correctly	Pending
Telemetry link	Open Mission Planner / run Python logger; confirm packets	Heartbeat received, CSV fields populated	Pending

Table 11. Bench-test protocol. Result column to be populated after hardware acquisition.

13.1 Bench Data Sheet

Parameter	M1	M2	M3	Notes
Battery voltage before test (V)				
5V rail idle (V)				
5V rail under servo load (V)				
Idle current draw (A)				
Full servo-load current (A)				
Motor no-load current at 25% throttle (A)				
GPS time to first 3D fix (s)				
Telemetry packet loss (%)				

Table 12. Bench data sheet — to be populated after hardware acquisition.

14. Virtual Testing and Simulation Plan

Virtual testing removes obvious design errors before hardware is risked. Three tools are planned: Fusion 360 for component fit checks and CG estimation, XFLR5 for airfoil and low-speed wing aerodynamic analysis, and ArduPilot SITL for autopilot logic verification and MAVLink logger testing without physical hardware.

Tool	Purpose	Input	Expected Output
Fusion 360	Component fit, CG envelope	CAD model + component mass estimates	Interference check, CG range

Tool	Purpose	Input	Expected Output
XFLR5	Airfoil and low-speed wing analysis	Airfoil, chord, span, speed range	Lift/drag curves, stall trends
OpenVSP	Parametric geometry study	Wing, fuselage, tail dimensions	Geometry model for analysis
ArduPilot SITL	Autopilot logic and MAVLink logger	Firmware, simulated vehicle, mission	Telemetry stream, CSV log

Table 13. Virtual testing tools and expected outputs.

15. Flight-Test Protocol

The maiden flight is a controlled experiment, not a demonstration. Its sole objective is to verify that the aircraft is controllable, balanced, and able to land safely. The propeller is installed only after all bench tests and an independent control-surface direction check.

Stage	Action	Abort Criteria
Pre-field	Verify firmware parameters, servo directions, failsafe, battery, CG, fasteners	Any mismatch or loose component
Range check	Confirm RC and telemetry link at increasing distance	RSSI/LQ below safe threshold
Launch rehearsal	Dry-run hand launch without powering motor	Poor grip or unclear role assignment
Maiden flight	Short stabilised flight, gentle turns, immediate landing	Excessive trim, oscillation, weak climb, or telemetry failure
Post-flight	Download logs, inspect battery, ESC, servos, airframe	Heat, cracks, loose joints, abnormal current draw

Table 14. Flight-test protocol stages and abort criteria.

16. Risk, Safety, and Regulatory Compliance

Risk	Cause	Control Measure
Propeller injury	Motor arms during bench work	Remove propeller for all configuration and testing
Battery fire	Short circuit, overcurrent, or damaged cell	Correct connectors, fuse where appropriate, safe charging
Loss of control	Wrong servo direction or V-tail mix	Two-person independent control-surface check before flight
Navigation failure	GPS/compass interference	GPS placement test and compass calibration before arming
Voltage brownout	Servo current exceeds BEC capacity	Servo-load bench test with adequate BEC rating
Flyaway	Failsafe misconfigured	Failsafe test without propeller; geofence and RTL planning
Regulatory breach	Flight in restricted airspace	Check local UAV regulations and obtain university approval

Table 15. Risk register with control measures.

17. Results Framework and Data Sheets

Empirical bench and flight results are not available at the time of this submission. This section is deliberately labelled as a results framework rather than a results claim. Leaving the tables blank and clearly marked as pending is academically stronger than fabricating data. All fields are structured to receive measured values directly after hardware acquisition.

Metric	Expected Source	Status	Interpretation Method
Battery voltage sag	MAVLink BATTERY_STATUS	Pending	Compare idle vs cruise vs climb
Cruise current draw	Power module log	Pending	Estimate endurance; validate battery sizing

Metric	Expected Source	Status	Interpretation Method
GPS lock time	Ground test log	Pending	Assess placement and interference level
Servo direction check	Bench observation + FC output	Pending	Confirm no reversal or binding
Airspeed vs groundspeed	VFR_HUD + GLOBAL_POSITION_INT	Pending	Quantify wind influence in flight
Mode transitions	Flight log	Pending	Verify manual, stabilise, RTL behaviour
Telemetry continuity	Packet count log	Pending	Assess radio link quality at range

Table 16. Results framework — to be populated after hardware acquisition and bench/flight testing.

18. Discussion and Resource Constraint Statement

The strongest engineering contribution in the current TriFlight scope is not a cosmetic airframe modification. It is the creation of a measurable UAV system design: a justified avionics architecture, a component selection process grounded in defined requirements, a working telemetry software implementation, and a staged validation plan that treats a maiden flight as a controlled experiment rather than a starting point.

A reference airframe can become a serious mechatronics project when the student engineer designs the avionics architecture, selects and justifies components, configures firmware, writes telemetry software, and documents a rigorous validation methodology. The distinction between an airframe purchase and an avionics engineering project is substantial and is maintained throughout this report.

Resource Constraint Statement

Physical implementation of the TriFlight avionics system was not completed. The reason is straightforward: the project lacked access to hardware funding. A minimum viable hardware set flight controller, GPS, power module, ESC, receiver, and telemetry radio represents an estimated expenditure of EUR 250–400. This was not available to the author as an independent student project without institutional laboratory allocation or external sponsorship.

All engineering work that does not require physical hardware has been completed: the research framework, requirements, architecture, trade studies, wiring design, firmware selection logic, Python MAVLink implementation, bench-test protocol, flight-test plan, and risk register. This work is original, documented, and ready for immediate practical implementation given access to components.

This report is submitted as evidence of engineering capability and structured thinking, not as a fabricated experimental result. The author is prepared to complete physical implementation immediately upon access to appropriate resources.

A defensible CV claim for this project: "Designed and documented a complete avionics modernisation for a fixed-wing UAV platform: power architecture, sensor integration, ArduPilot firmware configuration strategy, Python MAVLink telemetry logging, and staged validation methodology. Physical implementation pending hardware funding."

19. Development Roadmap

Phase	Duration	Deliverable	Status
1. Documentation and folder setup	2–3 days	BOM, wiring version control, safety checklist	Complete
2. Component finalisation	1 week	Final FC, GPS, ESC, receiver, telemetry, power module	Complete (design)
3. Wiring diagram	2–4 days	Clean system schematic and pin map	Complete (design)
4. Bench assembly	1 week	Electronics board assembled, no propeller	Pending hardware
5. Firmware setup	1 week	Parameters, modes, failsafe, servo mapping	Pending hardware
6. Telemetry logger	2–3 days	CSV logs from SITL or real FC	Code complete; SITL pending
7. Bench validation	1 week	Filled bench data tables and screenshots	Pending hardware
8. Maiden readiness review	2 days	Signed preflight checklist and go/no-go decision	Pending hardware
9. Field test	1 day + analysis	Short flight, logs, inspection notes	Pending hardware

Phase	Duration	Deliverable	Status
10. Paper revision	1 week	Updated results, plots, and conclusions	Pending hardware

Table 17. Development roadmap. Phases 1–3 and the Python logger are complete.

20. Attribution and Research Ethics

TriFlight is an educational and research-oriented avionics modernisation study. The external aircraft architecture used as the inspiration and reference platform is the Flightory Talon 1400, designed by Szymon Wojcik / Flightory. The reference platform and its original CAD files belong to their respective creator. TriFlight does not claim original authorship of the airframe geometry.

The student contribution described in this paper is the proposed avionics redesign, sensor expansion strategy, firmware configuration analysis, Python telemetry logging implementation, validation methodology, and research documentation. If this project is shown in a CV, portfolio, presentation, or GitHub repository, attribution must remain visible.

> **Recommended attribution wording:** "Based on the Flightory Talon 1400 platform for educational research; TriFlight work focused on avionics integration, sensor expansion, firmware configuration strategy, Python MAVLink telemetry logging, and validation methodology."

21. References

[1] ArduPilot Documentation. 'Using an Airspeed Sensor — Plane.' https://ardupilot.org/plane/docs/airspeed.html

[2] ArduPilot Documentation. 'RTL Mode (Return To Launch) — Plane.' https://ardupilot.org/plane/docs/rtl-mode.html

[3] ArduPilot Documentation. 'GeoFencing — Plane.' https://ardupilot.org/plane/docs/common-geofencing-landing-page.html

[4] ArduPilot Documentation. 'Automatic Landing — Plane.' https://ardupilot.org/plane/docs/automatic-landing.html

[5] MAVLink Developer Guide. 'MAVLink Common Message Set.' https://mavlink.io/en/messages/common.html

[6] MAVLink Developer Guide. 'Battery Protocol.' https://mavlink.io/en/services/battery.html

[7] PX4 User Guide. 'Airspeed Sensors.' https://docs.px4.io/main/en/sensor/airspeed

[8] PX4 User Guide. 'Assembling a Fixed-Wing.' https://docs.px4.io/main/en/assembly/assembly_fw

[9] PX4 User Guide. 'Controller Diagrams — Fixed-Wing TECS.' https://docs.px4.io/main/en/flight_stack/controller_diagrams

[10] OpenVSP. 'Learn More About OpenVSP.' https://openvsp.org/learn.shtml

[11] Flightory / Szymon Wojcik. Talon 1400 modular fixed-wing UAV platform. Used as credited reference baseline.

[12] pymavlink. MAVLink Python library. https://github.com/ArduPilot/pymavlink

Appendix A — Pin Mapping Template

FC Port	Assigned Device	Signal Type	Voltage	Notes
SERVO1	Left aileron	PWM	5V servo rail	Check deflection direction
SERVO2	Right aileron	PWM	5V servo rail	Opposite to SERVO1 for roll
SERVO3	Left V-tail	PWM	5V servo rail	V-tail mix: pitch + yaw
SERVO4	Right V-tail	PWM	5V servo rail	V-tail mix: pitch + yaw
SERVO5	ESC throttle	PWM/DShot	Signal + GND only	Remove prop during setup
UART1	GPS	Serial TX/RX	5V	TX/RX crossed between devices
I2C	Compass / airspeed	I2C SDA/SCL	3.3V or 5V	Check device voltage tolerance
UART2	Telemetry radio	MAVLink serial	5V	Baud rate must match radio
RCIN	ELRS receiver	CRSF/SBUS	5V	Bind and range test before flight
ADC/PM	Power module	Analog	VBAT scaled	Calibrate in Mission Planner

Table A1. Pin mapping template — confirm against selected flight controller datasheet.

Appendix B — Minimum Evidence Package

Before claiming this project as completed engineering work, the following evidence must be collected and appended to this document. Items marked Complete require no hardware.

Evidence Item	Purpose	Status
Clean wiring diagram (PDF or image)	Shows system architecture	Complete (design)
Component table with part numbers and cost	Shows justified selection process	Complete (design)
Firmware parameter export (.param file)	Proves configuration work	Pending hardware
MAVLink CSV log (SITL or real FC)	Proves telemetry logger works	Code complete; SITL pending
Bench voltage and current measurements	Proves electrical validation	Pending hardware
GPS and compass calibration screenshots	Proves sensor setup	Pending hardware
Signed preflight checklist	Proves safe operating procedure	Pending hardware
Flight log plots (voltage, current, altitude)	Proves experimental results	Pending hardware
Post-flight inspection photos	Proves airworthiness check	Pending hardware
Python logger on GitHub (public repository)	Demonstrates software authorship	Complete

Table B1. Minimum evidence package. Items marked Complete require no hardware acquisition.